

CrewConnect – Interview Document + Important Q&A;

Project Introduction

CrewConnect is a Django-based HR and Employee Management System. We have two major roles: HR and Employee.

Tech Stack: Python, Django, SQLite, HTML, CSS, Bootstrap, JavaScript, Django ORM, Django Authentication, and SMTP for email integration.

HR can manage employees, departments and designations, approve employee leave requests, generate payslips and view payroll reports. Employees can view their own profile, apply for leave, view leave history and calendar, and access their payslips.

Key Business Modules

- **Departments:** HR, Development, Testing and Finance can be maintained as department master data.
- **Designations:** Python Developer, Senior Python Developer, Software Tester, HR Executive and Accountant can be maintained as designation master data.
- **Employment Types:** Permanent, Contract, Part-time and Intern, implemented using Django model choices.
- **Authentication:** Django User with a OneToOne relationship to Employee.
- **Onboarding:** HR creates the employee, the system creates a User with an unusable password, generates a secure password setup token and sends an email through SMTP.
- **Leave:** Employee applies → Pending → HR approves/rejects → employee sees the updated status and calendar.
- **Payroll:** Monthly payslip with basic salary, allowances, deductions and net salary.
- **Reports:** HR selects month/year and gets payroll totals using Django ORM aggregation such as Sum.

Overall Workflow

Employee Onboarding → Authentication → Role Differentiation → Employee/HR Dashboard → Leave Management → Leave Approval → Payroll → Payslips → HR Reports

Important Interview Questions & Answers

Q: 1. What is CrewConnect?

A: CrewConnect is a Django-based HR and Employee Management System. It has two major roles, HR and Employee, and covers employee onboarding, authentication, leave management, leave approval, payroll, payslips and HR reporting.

Q: 2. What technologies did you use?

A: Python, Django, SQLite, Django ORM, HTML, CSS, Bootstrap, JavaScript, Django Authentication and SMTP for email integration.

Q: 3. Why did you choose Django?

A: Django provides built-in features such as authentication, ORM, URL routing, sessions, CSRF protection, templates and an admin interface. This helped us develop the HR system in a structured way.

Q: 4. How did you differentiate HR and Employee after login?

A: After successful authentication, we check whether an Employee record exists for the logged-in User. If it exists, we redirect to the Employee Dashboard; otherwise we redirect to the HR Dashboard.

Q: 5. Why did you use OneToOneField between User and Employee?

A: Django User handles authentication information while Employee stores business-specific employee information. OneToOne ensures one login account is associated with one employee profile.

Q: 6. Why did you use email as the username?

A: We use the employee email as the Django username because email is unique for each employee and is convenient for login. Since username is unique, we validate the email before creating the User.

Q: 7. How does employee onboarding work?

A: HR creates the employee. We create the Django User with an unusable password, create the Employee record, generate a password setup token and send a setup link to the employee through SMTP. The employee uses the link to create a password and then logs in.

Q: 8. Why do you use set_unusable_password()?

A: The HR user should not create or know the employee's password. The account initially has no usable password, and the employee creates their own password using the secure setup link.

Q: 9. How is the password setup link generated?

A: We encode the User ID using `urlsafe_base64_encode` and generate a token using Django's `default_token_generator`. The UID and token are included in the password setup URL.

Q: 10. What is the difference between console email backend and SMTP?

A: The console backend prints the email in the terminal and is useful during development. SMTP actually sends the email through an email server and is used when real email delivery is required.

Q: 11. Explain your leave workflow.

A: An employee submits a leave request. Its initial status is Pending. HR reviews it and either approves or rejects it. The employee can then view the updated status and leave calendar.

Q: 12. Why are Department and Designation separate models?

A: They are master data and can be reused by many employees. Keeping them in separate tables avoids repeated text and gives us proper ForeignKey relationships.

Q: 13. Why did you use ForeignKey for Department?

A: Many employees can belong to one department, so the relationship is many-to-one from Employee to Department. ForeignKey represents this relationship.

Q: 14. Why did you use on_delete=PROTECT for Department and Designation?

A: We don't want a department or designation to be deleted while employees are still using it. PROTECT prevents accidental deletion of referenced master data.

Q: 15. Why did you use CASCADE for User and Employee?

A: The Employee login profile depends on the User. If the User is intentionally deleted, the associated Employee record can also be removed.

Q: 16. How do you calculate net salary?

A: Net salary is calculated as Basic Salary + Allowances - Deductions. For example, 50,000 + 5,000 - 2,000 gives a net salary of 53,000.

Q: 17. How did you prevent duplicate payslips?

A: We added a database-level UniqueConstraint on employee, month and year. This means an employee can have only one payslip for a particular month and year.

Q: 18. Why use a database constraint instead of only checking in the view?

A: A view-level check can reduce duplicate records, but it is not a database guarantee. The unique constraint ensures the database itself rejects duplicates.

Q: 19. How does the payroll report work?

A: HR selects a month and year. We filter Payslip records for that period and use Django ORM aggregation functions such as Sum to calculate total basic salary, allowances, deductions and net salary.

Q: 20. What is aggregate()?

A: aggregate() calculates a summary value across a QuerySet and normally returns a dictionary. For example, Sum('net_salary') can calculate the total net salary.

Q: 21. What is annotate() and how is it different from aggregate()?

A: annotate() adds calculated information to each QuerySet object, while aggregate() calculates an overall summary for the QuerySet. For example, annotate can calculate annual salary for each employee, while aggregate can calculate total salary.

Q: 22. How do employees see only their own payslips?

A: We get the Employee object using the logged-in request.user and then filter Payslip objects using that employee. This prevents one employee from retrieving another employee's payslips through normal application flow.

Q: 23. What Django authentication functions did you use?

A: We use authenticate() to verify credentials, login() to create the authenticated session and logout() to end the session.

Q: 24. What is Django ORM?

A: Django ORM allows us to interact with database tables using Python models and QuerySets instead of writing SQL for every operation. We use it for filtering, creating, updating, relationships and aggregation.

Q: 25. Why SQLite?

A: SQLite is lightweight and doesn't require a separate database server, so it is convenient for development and testing. For a production system with higher concurrency and scale, we could move to PostgreSQL or SQL Server.

Q: 26. What frontend technologies did you use?

A: We use HTML and Django templates for structure and dynamic data, CSS for styling, Bootstrap for responsive UI, and JavaScript for client-side interactions.

Q: 27. How did you avoid repeating the sidebar and common layout?

A: We use Django template inheritance. Common layout is kept in base.html, and individual pages extend it using {% extends 'base.html' %} and {% block content %}.

Q: 28. What was one issue you faced during development?

A: We encountered unique constraint errors when creating a User with an email that already existed as a username. We also encountered a NameError when Decimal was used without importing it. We fixed these through validation and the required import.

Q: 29. What is the complete CrewConnect workflow?

A: The workflow is employee onboarding, authentication, role differentiation, employee/HR dashboard, leave management, leave approval, payroll and payslip generation, followed by HR reporting.

Q: 30. What would you improve in the future?

A: For production, I would consider PostgreSQL, stronger role/permission management, background email processing, audit logging, automated tests, better API integration if required, and production deployment with proper security and monitoring.

Quick Interview Revision

- **Project:** CrewConnect – HR and Employee Management System
- **Roles:** HR and Employee
- **Backend:** Python + Django
- **Database:** SQLite
- **ORM:** Django ORM
- **Authentication:** Django User + OneToOne Employee
- **Email:** SMTP
- **HR:** Employees, Departments, Designations, Leave Approval, Payroll, Reports
- **Employee:** Profile, Apply Leave, My Leaves, Leave Calendar, Payslips
- **Payroll:** Basic + Allowances - Deductions
- **Duplicate Prevention:** UniqueConstraint(employee, month, year)
- **Reporting:** filter by month/year + Sum aggregation
- **Final workflow:** Onboarding → Authentication → Leave → Approval → Payroll → Payslips → Reports